UNIVERSIDADE FRANCISCANA
Curso: Ciência da Computação – 2026-01.

Disciplina: Sistemas Operacionais
Professor: Ricardo Frohlich.

Nome (s): José Otávio R. Baggio
Data: 14/04/2026.

Revisão: Threads

1) Por que as threads são consideradas mais leves que processos? Relacione sua resposta com uso de memória e troca de contexto.

R:

Porque threads de um mesmo processo compartilham memória. O código, os dados globais e os arquivos abertos pertencem ao processo e ficam disponíveis para todas as suas threads — nada precisa ser duplicado. Quando um novo processo é criado, o sistema operacional precisa alocar um espaço de endereçamento completamente separado, copiar recursos, criar estruturas de controle próprias. Com threads, nada disso acontece. Trocar entre threads do mesmo processo é muito mais barato porque o espaço de endereçamento já é o mesmo — só precisam ser salvos o contador de programa, a pilha e os registradores daquela thread específica.

2) Explique o papel do contador de programa (PC) e da pilha (stack) em uma thread.

R:

Cada thread tem seu próprio **contador de programa (PC)** e sua própria **pilha (stack)**, mesmo compartilhando o restante do processo.

PC -> indica qual instrução a thread executará a seguir. Como threads podem estar em pontos diferentes do código ao mesmo tempo, cada uma precisa do seu próprio PC para saber onde parou quando perde a CPU e onde retomar quando volta a executar.

Pilha -> armazena as chamadas de função em andamento, as variáveis locais e os endereços de retorno daquela thread. Sem uma pilha própria, duas threads executando a mesma função simultaneamente misturariam suas variáveis locais, causando resultados incorretos.

3) Descreva dois benefícios do uso de multithreading em aplicações modernas.
R:

Responsividade: uma aplicação pode continuar respondendo ao usuário mesmo enquanto realiza uma tarefa demorada. Uma thread cuida da interface enquanto

outra processa dados em segundo plano.

Aproveitamento de múltiplos processadores: em sistemas com vários núcleos, threads podem executar verdadeiramente em paralelo, cada uma em um núcleo diferente, reduzindo o tempo total de execução de tarefas que podem ser divididas.

4) Dê um exemplo prático onde o uso de threads melhora a experiência do usuário e explique por quê.

R:

Um navegador web. Enquanto uma thread carrega e renderiza a página, outra executa scripts JavaScript e uma terceira trata os cliques do usuário. Se tudo fosse feito em sequência por um único fluxo de execução, a página travaria completamente durante o carregamento, o usuário não conseguiria nem rolar ou clicar em nada. Com threads, todas essas tarefas ocorrem simultaneamente e o navegador permanece sempre responsivo.

5) Explique como threads contribuem para o melhor aproveitamento de sistemas com múltiplos processadores.

R:

Em um sistema com vários núcleos, o sistema operacional pode escalonar threads diferentes do mesmo processo em núcleos diferentes ao mesmo tempo. Isso significa que um processo com quatro threads pode estar executando código em quatro núcleos simultaneamente, realizando em paralelo o que antes seria feito em sequência.

Sem threads, um processo só ocupa um núcleo por vez, os demais ficam ociosos para ele. Com multithreading o trabalho é dividido e distribuído entre os núcleos disponíveis, reduzindo drasticamente o tempo de execução de tarefas pesadas como processamento de vídeo, simulações e consultas em bancos de dados.

6) Diferencie threads de usuário e threads de kernel, destacando quem faz o gerenciamento, vantagens e desvantagens.

R:

Threads de usuário são gerenciadas por uma biblioteca no espaço do usuário (como a pthreads). O kernel não sabe que elas existem para ele, o processo inteiro é um único fluxo. A criação e a troca de contexto entre elas são muito rápidas porque não exigem chamada ao sistema.

A desvantagem é grave: se uma thread de usuário fizer uma chamada de sistema bloqueante (como leitura de disco), o kernel bloqueia o processo inteiro, paralisando todas as outras threads junto.

Threads de kernel são gerenciadas diretamente pelo sistema operacional. O kernel conhece cada thread individualmente e pode escaloná-las de forma independente, inclusive em núcleos diferentes. Se uma bloquear, as outras continuam executando normalmente.

A desvantagem é o custo: criar e trocar threads de kernel é mais lento pois envolve chamadas ao sistema operacional.

	Threads de Usuário	Threads de Kernel
Gerenciamento	Biblioteca do usuário	Sistema Operacional
Velocidade de criação	Rápida	Mais lenta
Bloqueio de I/O	Bloqueia todo o processo	Bloqueia só a thread
Paralelismo real	Não (kernel vê um processo)	Sim (kernel escalona cada um)

7) Explique o funcionamento:

a) modelo N:1 (many-to-one) e cite uma limitação importante.

R:

Vários threads de usuário são mapeadas para uma única thread de kernel. A biblioteca de threads gerencia toda a alternância entre elas no espaço do usuário, sem envolver o sistema operacional.

A limitação importante é que, como o kernel enxerga apenas uma thread, nunca há paralelismo real, mesmo em um sistema com vários núcleos, só uma thread executa por vez. Pior: se qualquer thread fizer uma chamada bloqueante, o kernel bloqueia a thread de kernel única e todas as threads de usuário param junto.

b) modelo 1:1 (one-to-one) e por que ele permite paralelismo real.

R:

Cada thread de usuário corresponde a uma thread de kernel. O sistema operacional conhece e gerencia cada thread individualmente.

Isso permite paralelismo real porque o kernel pode escalonar threads diferentes em núcleos diferentes ao mesmo tempo. Se uma thread bloquear em I/O, as demais continuam executando normalmente. É o modelo usado pelo Linux e Windows atualmente.

A desvantagem é que criar muitas threads tem custo maior, pois cada uma exige uma thread de kernel correspondente.

c) modelo M:N (many-to-many) e explique por que ele combina vantagens dos outros modelos.

R:

M threads de usuário são mapeadas para N threads de kernel, onde $N \leq M$. O sistema operacional gerencia as threads de kernel e a biblioteca de threads distribui as de usuário entre elas.

Combina as vantagens dos dois modelos: é possível criar muitas threads de usuário com baixo custo, e ao mesmo tempo o kernel tem threads suficientes para permitir paralelismo real em múltiplos núcleos. Se uma thread bloquear, o sistema pode redirecionar outra thread de usuário para uma thread de kernel disponível. É o modelo mais flexível, porém o mais complexo de implementar.

8) Em qual modelo ocorre o problema em que, se uma thread bloquear, todas as outras também param? Explique o motivo.

R:

No N:1(Many-to-one). No modelo **N:1**. O motivo é que todas as threads de usuário compartilham uma única thread de kernel. Quando o kernel recebe uma chamada bloqueante vinda desse processo, ele bloqueia a única thread de kernel que possui — e como todas as threads de usuário dependem dela para executar, nenhuma consegue avançar até que o bloqueio seja liberado. O kernel simplesmente não sabe que existem outras threads esperando, pois toda a gestão delas é invisível para ele.

9) Para que servem os métodos sleep() e join()? Explique com exemplos conceituais.

R:

sleep() faz uma thread pausar sua execução por um tempo determinado, liberando voluntariamente a CPU para outras threads. A thread fica em estado de espera até o tempo acabar e então volta à fila de prontos.

Exemplo conceitual: uma thread que monitora temperatura a cada 5 segundos chama sleep(5000) após cada leitura, evitando consumir CPU continuamente sem necessidade.

join() faz uma thread esperar que outra termine antes de continuar. Quem chama join() fica bloqueado até que a thread alvo finalize sua execução.

Exemplo conceitual: a thread principal divide um cálculo pesado em quatro threads trabalhadoras e chama join() em cada uma. Só depois que todas terminam ela soma os resultados parciais e exibe o resultado final. Sem o join(), a thread principal poderia tentar ler os resultados antes de estarem prontos.

10) O que significa dizer que um método é synchronized? Explique a importância em aplicações com múltiplas threads?

R:

Significa que apenas uma thread por vez pode executar aquele método. Quando uma thread entra em um método synchronized, ela adquire um "bloqueio" (lock) sobre o objeto. Qualquer outra thread que tente entrar no mesmo método ficará esperando até o bloqueio ser liberado.

A importância em aplicações multithreaded é evitar **condições de corrida** — situações onde duas threads acessam e modificam um dado compartilhado ao mesmo tempo, produzindo resultados incorretos e imprevisíveis.

Exemplo: duas threads incrementando um contador compartilhado simultaneamente. Sem sincronização, ambas podem ler o valor 5 ao mesmo tempo, ambas calculam 6 e ambas escrevem 6, o resultado final é 6 em vez do correto 7. Com synchronized, uma espera a outra terminar antes de acessar o contador, garantindo a consistência dos dados.

11) Explique o papel das funções pthread_create, pthread_join e pthread_exit.

R:

pthread_create: cria uma nova thread. Recebe como parâmetros um identificador para a thread, atributos de configuração, a função que a thread vai executar e os argumentos dessa função. Após a chamada, a thread nova passa a existir e compete pela CPU junto com as demais.

pthread_join: faz a thread chamadora esperar que uma thread específica termine. Funciona como o join(), garante sincronização e permite coletar o valor de retorno da thread que terminou. Sem pthread_join, a thread principal pode encerrar o processo antes das threads filhas terminarem, destruindo todo o trabalho em andamento.

pthread_exit: encerra a thread que a chamou, sem encerrar o processo inteiro. Permite que a thread retorne um valor que pode ser capturado por quem chamou pthread_join. Se a thread principal chamar pthread_exit em vez de retornar do main(), o processo continua vivo enquanto houver outras threads executando.

12) Explique a diferença entre concorrência e paralelismo, relacionando com o uso de threads.

R:

Concorrência é a capacidade de lidar com várias tarefas ao mesmo tempo, mas não necessariamente executando-as simultaneamente. Em um sistema com um único núcleo, o sistema operacional alterna rapidamente entre threads, dando a ilusão de que tudo acontece ao mesmo tempo. As tarefas progridem de forma intercalada, não simultânea.

Paralelismo é a execução de fato simultânea de múltiplas tarefas, exigindo múltiplos núcleos. Duas threads realmente executam ao mesmo tempo, cada uma em um núcleo diferente.

Em um sistema de núcleo único, múltiplas threads funcionam de forma **concorrente**, o escalonador alterna entre elas rapidamente. Em um sistema multinúcleo com o modelo 1:1, as threads podem ser **paralelas** de verdade, o kernel as distribui entre os núcleos disponíveis.

	Concorrência	Paralelismo
Núcleos necessários	1 ou mais	Obrigatoriamente mais de 1
Execução	Intercalada	Simultânea
Exemplo com threads	RR em núcleo único	4 threads em 4 núcleos